



## **GLA UNIVERSITY, MATHURA**

**Synopsis of**

***Video Transcoding Website***

**Submitted by**

**Samriddhi Bansal(2415001397)**

**Kajal Saraswat (2415000746)**

**Monika Chanchal(2415000991)**

**Meenakshi(2415000967)**

**Under the supervision of**

***Md Wasif Arshad***

**Department of Computer Engineering Application**

## 1. Problem Statement

Video Transcoding Website — A web-based platform that allows users to upload video files and automatically convert (transcode) them into multiple resolutions (1080p, 720p, and 480p), making it easy to store, share, preview, and download videos across different devices and network speeds, without requiring any technical knowledge of video formats or third-party desktop software.

## 2. Problem Statement Description

In the present digital era, video has become one of the most widely used forms of content on the internet, whether for education, entertainment, communication, or business. However, videos captured on modern devices are often of very high resolution and large file size, which creates practical difficulties when it comes to sharing, uploading, or streaming them over the internet.

A large, uncompressed video file takes a long time to upload or download, especially for users with limited bandwidth. Not every user has access to high-speed internet or high-end devices capable of smoothly playing high-resolution video. At the same time, manually compressing or converting videos into different resolutions requires thirdparty desktop software, technical knowledge of codecs and formats, and is inconvenient for an average user who simply wants a smaller, shareable version of their video.

There is a need for a simple, web-based platform where a user can upload a video once, and the system automatically takes care of converting it into multiple resolutions on demand, storing it reliably, and making it available for instant preview or download — without requiring the user to install or understand any video-processing software.

## 3. Objective and Scope

### 3.1 Objectives

1. Develop a full-stack web application that allows users to upload video files and automatically transcode (convert and compress) them into multiple resolutions.
2. Enable conversion of uploaded videos into standard resolutions (1080p, 720p, and 480p) so that users can choose the quality best suited to their internet speed and device.
3. Perform video compression on demand — only when a specific resolution is requested — to optimize server resource usage instead of pre-processing every resolution at upload time.
4. Store original and processed videos on cloud storage and provide publicly accessible streaming URLs for instant playback in the browser.

### 3.2 Scope

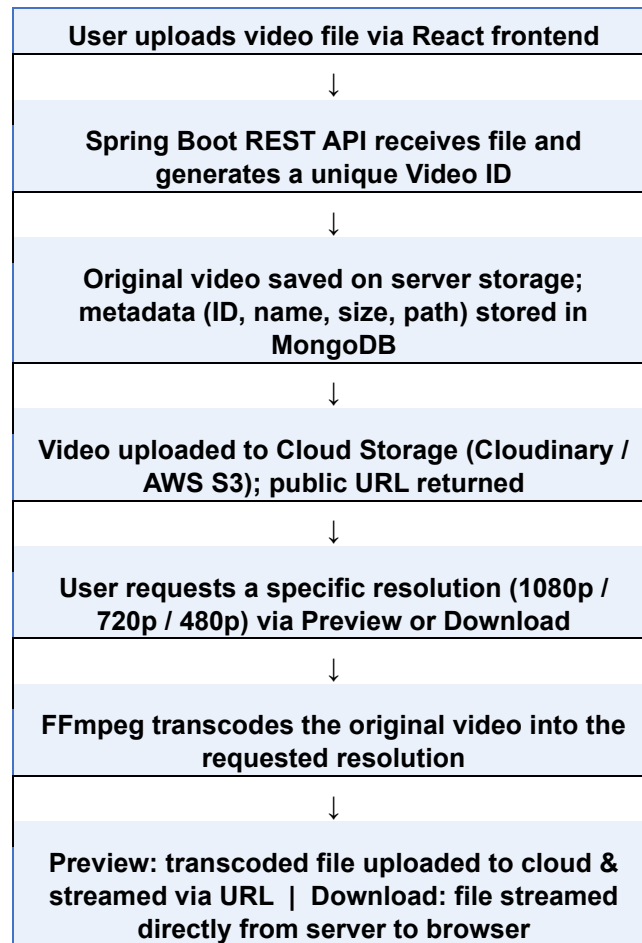
The scope of the project covers the complete lifecycle of a video within the system: upload, temporary local storage, permanent cloud storage, on-demand multi-resolution transcoding using FFmpeg, metadata management using MongoDB, and delivery to the end user through either an in-browser preview or a direct download.

The current scope focuses on the core transcoding pipeline and does not include user authentication/accounts, which is identified as an item for future enhancement. The project is intended as a functional prototype demonstrating an efficient, resourceconscious approach to on-demand video transcoding, rather than a production-scale streaming service such as YouTube or Netflix.

## 4. Proposed Methodology

### 4.1 Process Flow

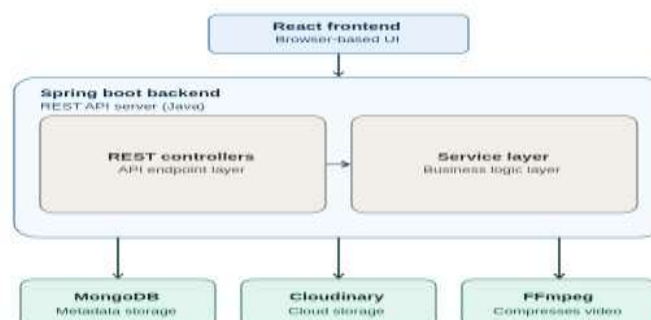
The following flow describes how data and control move through the system, from the moment a user uploads a video to the moment they receive a playable or downloadable file:



## 4.2 Development Phase

- Phase 1 — Requirement analysis and system design
- Phase 2 — Backend setup: Spring Boot project structure, MongoDB connectivity
- Phase 3 — Upload module: REST API to receive and store video files
- Phase 4 — Cloud storage integration for permanent, streamable storage
- Phase 5 — FFmpeg-based transcoding module for multi-resolution conversion
- Phase 6 — Preview and Download REST APIs
- Phase 7 — Integration with the existing React frontend
- Phase 8 — Testing, debugging, and documentation

## 4.3 System Architecture



## 5. Software Requirements

- **Backend:** Java (JDK 17+), Spring Boot Framework, Maven
- **Database:** MongoDB
- **Frontend:** React.js, React Router, Axios, TailwindCSS, Vite
- **Video Processing:** FFmpeg
- **Cloud Storage:** Cloudinary (current) — with AWS S3 integration planned for future scaling
- **IDE/Tools:** IntelliJ IDEA, Visual Studio Code, Postman (for API testing)

## 6. Future Work

1. Integration with AWS (S3 for storage, EC2 for deployment) to make the application publicly accessible and cloud-scalable.
2. Implementation of adaptive streaming using HLS (HTTP Live Streaming) for smoother playback based on network conditions.
3. Automatic thumbnail generation for uploaded videos.
4. User authentication and account system to maintain a history of uploaded/transcoded videos.
5. An analytics dashboard to track views, downloads, and most-used resolutions.
6. Progress-tracking for transcoding using WebSockets, instead of a static loading indicator.

## 7. Project Schedule (Gantt Chart)

The project is planned to be completed over an 8-week period, as illustrated below:

| Task                                       | W1 | W2 | W3 | W4 | W5 | W6 | W7 | W8 |
|--------------------------------------------|----|----|----|----|----|----|----|----|
| Requirement Analysis & Planning            |    |    |    |    |    |    |    |    |
| Spring Boot Project Setup & DB Connection  |    |    |    |    |    |    |    |    |
| Video Upload Module Development            |    |    |    |    |    |    |    |    |
| Cloud Storage Integration (Cloudinary/AWS) |    |    |    |    |    |    |    |    |
| FFmpeg Transcoding Module                  |    |    |    |    |    |    |    |    |
| Preview & Download API Development         |    |    |    |    |    |    |    |    |
| Frontend Integration with New Backend      |    |    |    |    |    |    |    |    |
| Testing, Debugging & Documentation         |    |    |    |    |    |    |    |    |

## 7. Conclusion

This project aims to build a practical, resource-efficient video transcoding platform that solves a real and common problem — the inconvenience of sharing and viewing large video files across varying network speeds and devices.

The on-demand transcoding approach, combined with cloud-based storage, makes the system both efficient and scalable, laying a solid foundation for future enhancements such as AWS integration, adaptive streaming, and user account management.

## 8. References / Bibliography

- [1] The Essential Guide to Video Processing : AL BOVIK
- [2] FFmpeg Documentation, FFmpeg Project, 2024. [Online]. Available: <https://ffmpeg.org/documentation.html>
- [3] Cloudinary Developer Documentation, Cloudinary Ltd., 2024. [Online]. Available: <https://cloudinary.com/documentation>
- [4] MongoDB Manual, MongoDB Inc., 2024. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [5] AWS S3 Developer Guide, Amazon Web Services Inc., 2024. [Online]. Available: <https://docs.aws.amazon.com/s3/>
- [6] Spring Boot Reference Documentation, VMware Tanzu / Broadcom, 2024. [Online]. Available: <https://docs.spring.io/spring-boot/documentation.html>
- [7] D. Rodriguez, "Introduction to Audiovisual Transcoding, Editing, and Color Analysis with FFmpeg," Programming Historian, 2018. [Online]. Available: <https://programminghistorian.org/en/lessons/introduction-to-ffmpeg>
- [8] shivam121q1, "Transcoder," GitHub repository, 2026. [Online]. Available: <https://github.com/shivam121q1/Transcoder>